

App Dev Project Report

Author

Name: Jaiveer *Trivedi*

Roll Number: 24f2004666

Student Email: 24f2004666@ds.study.iitm.ac.in

About me: I am a third year student pursuing a dual degree(GTU-BE in CSE + IITM BS degree). Currently exploring traditional development as well as emerging technologies in AI/ML.

Project Details

Project Title: Trekking Management Application

Problem Statement:

To build a Trekking Management Application web application that allows Admin, Trek Staff, and Users (Trekkers) to interact with the system based on their roles.

Approach:

I used Flask with Blueprints to separate the routes by role(auth, admin, user, staff), SQLAlchemy models for the database, and Flask-Login for sessions and role based access, with Werkzeug for password hashing. The booking logic checks trek status and slots, prevents duplicate bookings, and uses soft-cancel to keep history. The frontend uses HTML, Jinja2 and Bootstrap with no JavaScript for core features.

AI/LLM Usage

The extent of AI/LLM (Claude) usage is around 10-15%, limited to code suggestions and documentation formatting.

All final implementation logic, debugging and integration were done manually.

Technologies and Frameworks Used

- **Flask** -> for render_template, blueprint -> redirect, routes (backend)
 - **Flask-SQLAlchemy** -> for db management
 - **Flask-Login** -> for secure login and role based access
 - **Werkzeug** -> for hashing and password check
 - **Jinja2** -> for HTML templating.
 - **HTML** — frontend
 - **Bootstrap** — for frontend styling
-

DB Schema Design

Admin table

Column	Data type	Constraint
admin_id	Integer	Primary Key, Autoincrement
username	String(100)	Unique, Not null
password	String(255)	Not null

Staff table

Column	Data type	Constraint
staff_id	Integer	Primary Key, Autoincrement
name	String(100)	Not null
phone_number	String(15)	Not null
status	String(20)	Not null, default='active'
number_of_treks_completed	Integer	Not null, default=0
email	String(100)	Unique, Not null
password	String(255)	Not null
is_approved	Boolean	Not null, default=False

Trek table

Column	Data type	Constraint
trek_id	Integer	Primary Key, Autoincrement
trek_name	String(100)	Not null
location	String(100)	Not null
difficulty	String(20)	Not null
duration	Integer	Not null
available_slots	Integer	Not null
assigned_staff_id	Integer	Foreign key=staff.staff_id, Nullable
status	String(20)	Not null
start_date	DateTime	Not null
end_date	DateTime	Not null
price	Float	Not null

Users table

Column	Data type	Constraint
user_id	Integer	Primary key, Autoincrement
name	String(100)	Not null
email	String(100)	Unique, Not null
password	String(255)	Not null
is_blacklisted	Boolean	default=False

Booking table

Column	Data type	Constraint
booking_id	Integer	Primary Key, Autoincrement
user_id	Integer	Foreign Key="users.user_id", Not null
trek_id	Integer	Foreign Key="treks.trek_id", Not null
booking_date	DateTime	Not null
status	String(20)	Not null
payment_status	String(20)	Not null, default='pending'

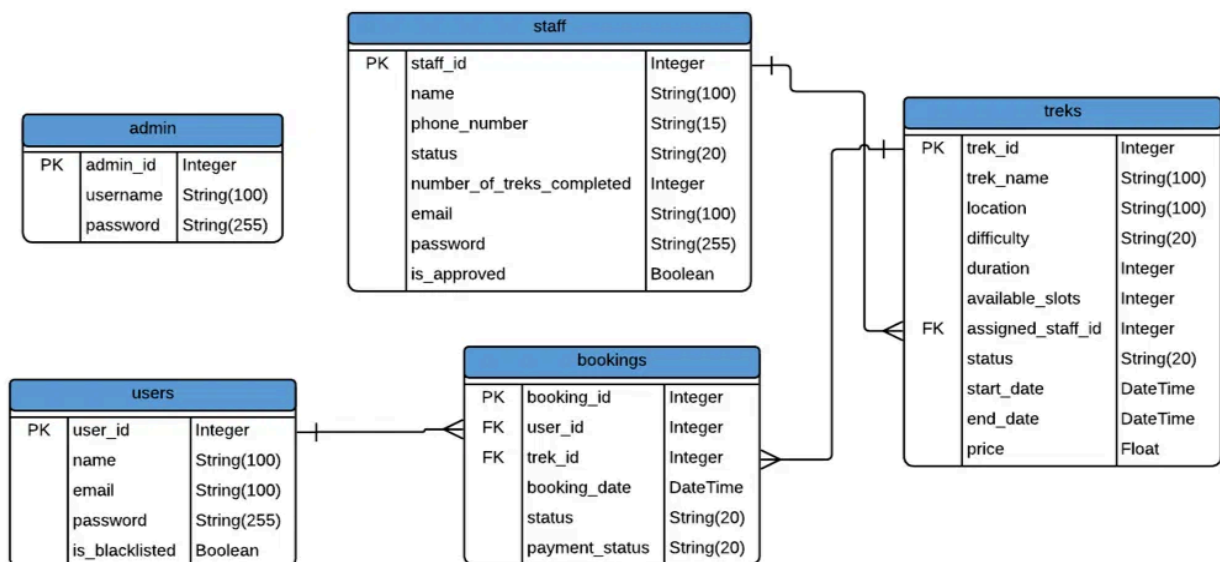
Relationships:

Staff -> One to many with Trek

Trek -> One to many with Booking

User -> One to many with Booking

ER DIAGRAM -> LUCIDCHART



Architecture and Features

Visual representation of project directory

/trekking-management-system

 /instance (trekking.db — auto-created by SQLAlchemy)

 /models (__init__.py + 5 models)

 /routes (4 routes)

 /templates (HTML templates)

 /static (trek images)

 .gitignore

 app.py

 README.md

 requirements.txt

Implemented Features:

Authentication: role-based login for Admin, Staff and User. Staff can log in only after Admin approval. Admin is pre-created (no registration).

Admin: dashboard with live counts, create/edit/delete treks, approve, unapprove and delete staff; assign staff to treks, search treks, users and staff by name or ID, blacklist/unblacklist users, view all bookings and trekking history.

Staff: edit profile, access only assigned treks, dashboard with assigned-trek and registered-trekker counts, update slots and trek status, view and manage participant list, mark treks started/ongoing/completed.

User: edit profile, view open treks, filter by difficulty and location, book treks with duplicate and full/closed prevention, cancel bookings, view booked treks, status and trekking history.

Additional features: offline payment tracking via a payment_status toggle managed by Admin. Flask-Login-based security with session protection and role-restricted routes.

Video:

<https://drive.google.com/file/d/1IPMqAqLAWftx0PELTpIJrgMJtcimAC1c/view?usp=sharing>